

Data Analytics — Project 3

SQL Data Analysis

Industrial Training Kit · Batch 2026 · Powered by DecodeLabs

Prepared by: Fatema Kamal (Meem)

This document reports the SQL data analysis performed on the provided orders dataset (1,200 records), covering querying, filtering, sorting, grouping, aggregation, and business insight extraction as required by the Project 3 brief.

1. Project Overview

Project 3 of the DecodeLabs Industrial Training Kit asks trainees to step into the role of a Data Analyst and use SQL to extract insights from a raw dataset. The task moves beyond simply viewing a spreadsheet — it requires writing structured, declarative queries that filter, group, and aggregate data into actionable business intelligence.

Goal

Use SQL queries to extract insights from the provided dataset.

Key Requirements Covered

- Write SELECT queries
- Filter rows using WHERE (equality, comparison, pattern matching, NULL handling, ranges, set membership)
- Sort results using ORDER BY
- Group records using GROUP BY
- Perform aggregations: COUNT, SUM, AVG, MIN, MAX
- Filter grouped/aggregated results using HAVING (bonus)
- Calculate percentage contribution of categories (bonus)

Dataset Summary

Attribute	Detail
Source file	Dataset_for_Data_Analytics.xlsx
Total records	1,200 orders
Columns	14 (OrderID, Date, CustomerID, Product, Quantity, UnitPrice, ShippingAddress, PaymentMethod, OrderStatus, TrackingNumber, ItemsInCart, CouponCode, ReferralSource, TotalPrice)
Date range	2023-01-01 to 2025-06-30
Products	Monitor, Phone, Tablet, Chair, Printer, Laptop, Desk
Order statuses	Shipped, Cancelled, Returned, Delivered, Pending
Payment methods	Debit Card, Online, Credit Card, Gift Card, Cash
Missing data	CouponCode has 309 nulls — a legitimate 'no coupon used' state, not a data quality issue

Tools Used

The dataset was loaded into a local SQLite database (orders.db) using Python (pandas + sqlite3), and all analysis was performed with standard ANSI-compatible SQL so the same queries port cleanly to MySQL, PostgreSQL, or SQL Server with only minor date-function adjustments.

2. Methodology: How the Query Engine Was Approached

As emphasized in the training brief, SQL is a **declarative** language: each query states the desired result, and the database engine decides how to fetch it. The logical execution order does not match the order in which SQL is written — this project's queries were built with that execution order in mind.

Step	Clause	Purpose
1	FROM / JOIN	Locate the data source (orders table)
2	WHERE	Filter individual rows before any grouping
3	GROUP BY	Bucket the filtered rows into categories
4	HAVING	Filter the aggregated buckets
5	SELECT	Choose output columns and compute aliases
6	ORDER BY	Sort the final result set (can reference SELECT aliases)

This ordering explains a common pitfall referenced in the training material — the 'Alias Trap': an alias created in SELECT cannot be used inside a WHERE clause on the same query level, because WHERE executes before SELECT. Every query in this solution respects that rule; aliases are only reused in ORDER BY, which runs last.

Database Schema (as loaded into SQLite)

Table: **orders** — 1,200 rows, 14 columns (see Section 1 for full column list).

3. Solution Structure — solution.sql

The full solution (32 queries) is organized into six sections inside **solution.sql**, progressing from basic retrieval to advanced business analysis:

#	Section	Queries	Covers
1	Basic SELECT	4	Column selection, aliasing, DISTINCT
2	Filtering (WHERE)	9	Equality, comparison, AND/OR, LIKE, IS NULL, BETWEEN, IN
3	Sorting (ORDER BY)	3	Single/multi-column sort, sorting by computed alias
4	Grouping & Aggregation	7	COUNT, SUM, AVG, MIN, MAX, monthly trends
5	HAVING	3	Filtering grouped/aggregated results
6	Advanced / Bonus	6	% contribution, coupon impact, customer ranking, risk & conversion rates

Example query (percentage contribution, Section 6.1):

```
SELECT Product, SUM(TotalPrice) AS product_revenue,  
       ROUND(100.0 * SUM(TotalPrice) / (SELECT SUM(TotalPrice) FROM orders), 2)  
       AS pct_of_total_revenue  
FROM orders  
GROUP BY Product  
ORDER BY pct_of_total_revenue DESC;
```

This pattern (aggregate ÷ correlated-subquery total) is the standard way to compute share-of-total in pure SQL without a separate calculation step.

4. Key Findings

4.1 Revenue Overview

Metric	Value
Total orders	1,200
Total revenue	\$1,264,761.96
Average order value	\$1,053.97
Top revenue product	Chair — \$195,620.11 (15.47% of total revenue)

4.2 Order Status Breakdown

Status	Orders	% of Orders
Cancelled	250	20.83%
Returned	247	20.58%
Pending	237	19.75%
Shipped	235	19.58%
Delivered	231	19.25%

Only ~19% of orders reach 'Delivered', while over 41% end in Cancelled or Returned. This combined problem rate is a clear signal for further root-cause analysis on fulfillment and product quality.

4.3 Product Risk Analysis (HAVING/CASE-based)

Monitor (43.56%), Tablet (43.02%), and Laptop (42.77%) carry the highest combined cancellation + return rates of all products, making them priority candidates for a fulfillment or quality-control review.

4.4 Coupon Impact

Coupon Group	Orders	Avg Order Value
FREESHIP	313	\$1,070.41
SAVE10	286	\$1,065.87
No Coupon	309	\$1,043.37
WINTER15	292	\$1,035.90

Average order value is fairly consistent across coupon groups (within ~3% of each other), suggesting coupons in this dataset are not strongly correlated with basket size.

4.5 Best-Converting Referral Source

Email referrals have the highest delivery-completion rate at 24.0%, making Email the most reliable acquisition channel by fulfillment outcome among the five tracked sources (Instagram, Referral, Email, Facebook, Google).

4.6 Top Customers

Customer ID	Total Spend	Orders
C38840	\$5,723.23	2
C57276	\$3,456.40	1
C67260	\$3,390.80	1

5. Deliverables & File Structure

File	Description
solution.sql	Full SQL solution — 32 queries across 6 sections (see Section 3)
orders.db	SQLite database built from the dataset (table: orders)
Project3_SQL_Results.xlsx	Every query's output, one sheet per query, plus an Index sheet
Project3_Documentation.pdf	This report
README.md	GitHub-ready project overview and setup instructions

6. Conclusion

This project demonstrates the full analytical arc required by the DecodeLabs Project 3 milestone: writing SELECT statements, filtering with WHERE across multiple operator types, sorting with ORDER BY (including sorting by computed aliases), grouping and aggregating with COUNT/SUM/AVG/MIN/MAX, filtering aggregated groups with HAVING, and layering on percentage-contribution and rate-based analysis for genuine business insight.

The result is a query set that bridges a 1,200-row raw dataset and specific, decision-ready answers — the 'digital lens' the brief asked trainees to build — and a portfolio artifact ready to publish to GitHub.